君正®

Zeratul T23 开发指南

Date: June.2023

北京君正集成电路股份有限公司
Ingenic Semiconductor Co., Ltd.

君正®
Zeratul T23 开发指南
Copyright © Ingenic Semiconductor Co. Ltd 2019. All rights reserved.

Release history

Date	Revision	Change
Jun.2023	1.0	First release
Jan.2024	1.1	更新烧录文档命名

Disclaimer

This documentation is provided for use with Ingenic products. No license to Ingenic property rights is granted. Ingenic assumes no liability, provides no warranty either expressed or implied relating to the usage, or intellectual property right infringement except as provided for by Ingenic Terms and Conditions of Sale.

Ingenic products are not designed for and should not be used in any medical or life sustaining or supporting equipment.

All information in this document should be treated as preliminary. Ingenic may make changes to this document without notice. Anyone relying on this documentation should contact Ingenic for the current documentation and errata.

北京君正集成电路股份有限公司

地址: 北京市海淀区西北旺东路 10 号院东区 14 号楼君正大厦
电话:(86-10)56345000
传真:(86-10)56345001
Http: //www.ingenic.com

目录

1. 基础概念与架构 1
1.1 摘要 1
1.2 产品架构 1
1.2.1 套装类产品架构 1
1.2.2 单品类产品架构 2
2. 开发流程 3
3. 编程规范 4
4. 搭建 Camera 基础系统 5
4.1 系统分区编译 5
4.2 如何制作并进入 Normal u-boot 5
4.3 调试阶段文件更新与固件烧录 5
4.3.1 文件更新 5
4.3.2 文件系统环境下烧录 6
4.3.3 u-boot 环境下烧录 6
4.3.4 TF 卡烧录 6
4.3.5 烧录地址 6
4.4 恢复与升级 7
4.4.1 恢复初始状态与设置 7
4.4.2 升级 7
5. MCU_WDT 功能 9
5.1 MCU_WDT 是什么功能？主要用来解决什么问题 9
5.2 MCU_WDT 功能真值表 9
5.3 MCU_WDT 的使用 9
6. Camera 快速启动优化 10
6.1 尽快运行主程序 10
6.2 缩短 rootfs 的加载时间 11
6.3 IRCUT 和 IRLED 的处理 11
6.3.1 TAG ENV 处理方法 11
6.3.2 IRCUT IRLED GPIO 设置 12
6.3.3 内核操作方法 13
6.4 ADC 取值和日夜切换阈值 13
6.5 NRVBS 说明 14
6.6 快速启动分辨率和 nrvbs 设置方法 15
6.7 帧率的修改 15

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

6.8 系统内存以及优化 16
6.8.1 系统内存说明 16
6.8.2 系统内存优化 17
6.9 分区信息如何调整 18
6.9.1 正常系统下分区调整 18
6.9.2 Recovery 系统下的分区调整 18
6.10 图像尺寸缩放说明 19
6.10.1 图像缩放配置 19
6.10.2 图像缩放裁剪计算方法 20
6.11 ISP 驱动介绍 21
6.11.1 回调函数 21
6.11.2 分辨率和 buffer 配置 22
6.11.3 API 说明 23
6.11.4 编译 23
6.11.5 丢帧和 debug 23
6.11.6 帧率切换 24
6.11.7 出图时间优化 24
6.11.8 日夜切换 25
6.11.9 图像镜像翻转 26
6.12 TAG 分区内容详解 27
6.12.1 TAG 分区数据更新方法 28
6.12.2 IQ 效果文件更新验证 28
7. 常见问题 29

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

1. 基础概念与架构

1.1 摘要

Zeratul 是基于君正 T 系列芯片的电池类 IPC 开发平台，通过 Zeratul，开发者可以快速开展电池类产品开发。

Zeratul 基于 ISVP（君正 T 系列智能视频开发平台），并封装了电池类 IPC 特殊的功能接口，比如 WIFI 连接、电源管理、关机唤醒等等功能。

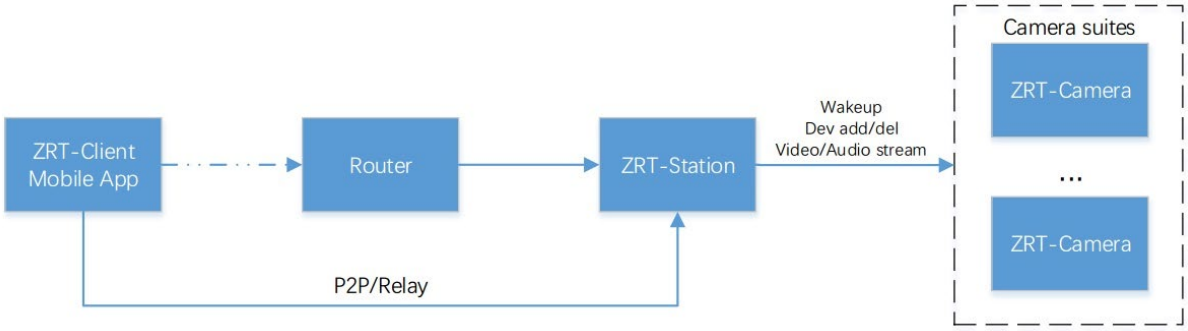
ISVP 与 Zeratul 的关系：ISVP 中包含了 toolchain，基础操作文档（如 TF 启动卡制作教程），以及 libimp 的 API（IPC 产品音视频业务）及 doc 等等基础资源。Zeratul SDK 在此之上，拓展了电池类 IPC 所需的功能和应用，例如开关机控制，WIFI 操作等等。

1.2 产品架构

电池类 IPC 产品一般分为两种形态:

- 套装：产品由一个 Station+若干 Camera 组成
- 单品：产品为一个 Camera，直接链接路由器

1.2.1 套装类产品架构



如上图所示，Zeratul 在套装产品形态上由三部分构成：

- **Camera**

摄像头，由电池供电，如果没有用户连接操作的话通常处于待机状态——主控 TxxZ 关机，WIFI 保持连接。

在待机状态下，有以下几种唤醒方式：Client 唤醒（WIFI）、PIR 唤醒、按键唤醒。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

- **Station**

WIFI 中继，由 AC 供电，可连接多个 Camera，提供 Camera 唤醒服务以及码流中继功能。Internet 接入口为以太网接口，通过 dhcp 获取 IP——即插入家用路由器即可上网。

Station 和 Camera 之间有两种关系状态：连接/非连接。默认情况下，Camera 和 Station 之间是非连接状态，在用户长按“SYNC”按键后，Station 进入准备连接的状态，Camera 同样也有“SYNC”按键，长按进入准备连接状态。Station 和 Camera 在合理范围内，同时处在准备连接状态时，会自动匹配进行连接（这个过程类似于 WIFI 的 WPS 连接功能）。如果要解除连接状态，需要在 Client 端操作。

- **Client**

手机端 App，用户功能与一般家用 IPC 产品类似。

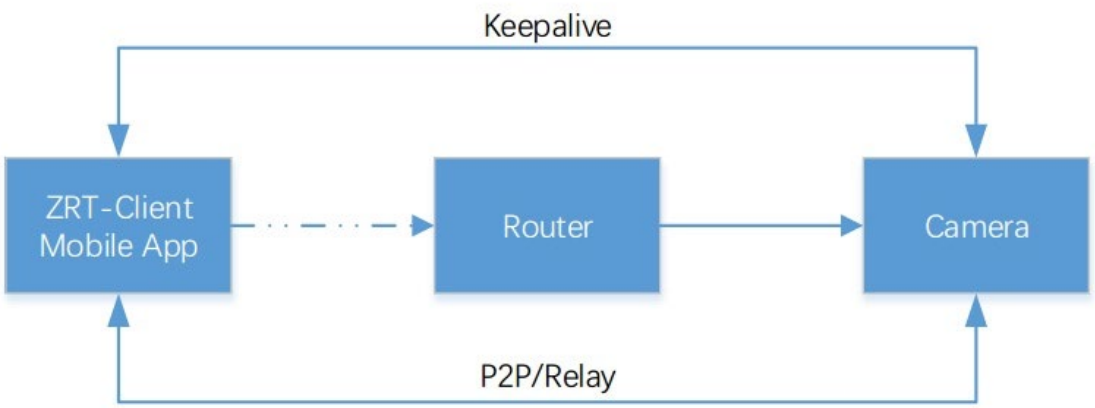
➢ Client 与 Station 的关系

Client 与 Station 之间有两种关系状态：非绑定/绑定。在客户第一次使用产品时，Client 和 Station 是没有绑定的，需要 Client 和 Station 连接在同一个局域网（Client 通过 WIFI 连接路由器、Station 通过 ETH 连接路由器），再进行绑定流程，成功后，Client 与 Station 建立联系，可以进行接下来的操作。Client 可以主动解除与 Station 的绑定。 在 Station 被绑定之后 Client 可以通过 Station 查看到与此 Station“连接”的 Camera，即可以进行这些 Camera 的操作。

➢ Client 与 Camera 的关系

Client 通过 Station 获取到 Camera 的信息——比如 SN，可以与 Camera 建立通信通道，就像普通家用 IPC 一样。区别于一般家用 IPC 的是，在操作（比如查看视频图像）完成后，需要关闭 Camera，使其进入待机状态，下次需要操作时，需要唤醒 Camera 后，才能与其进行连接。

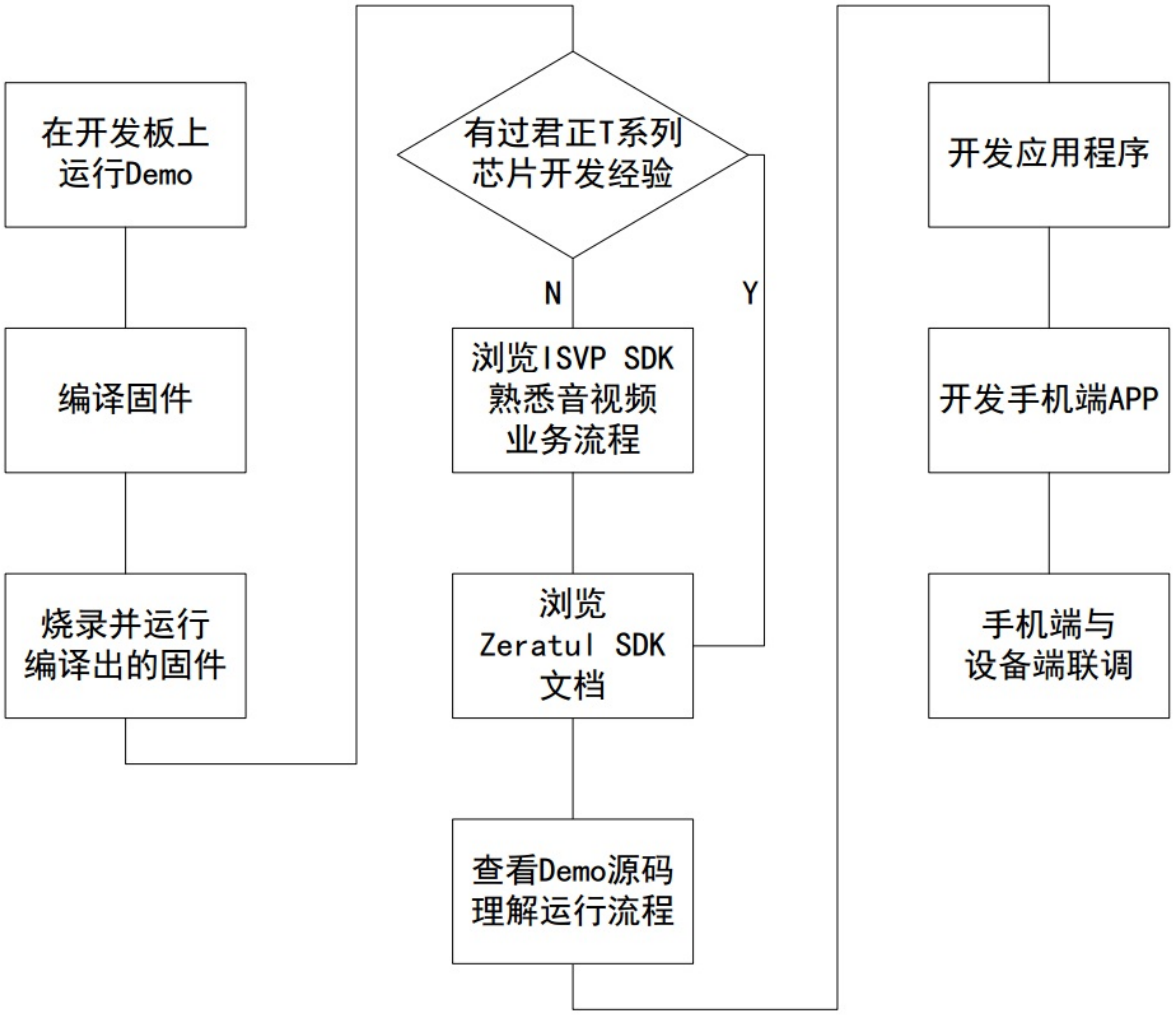
1.2.2 单品类产品架构



对于单品类型的产品，则直接由 Camera 接入路由器，因此也需要由 Camera 发送心跳包以 Keep Alive。Camera 与 Client 的角色功能与套装类产品类似。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

2. 开发流程



Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

3. 编程规范

由于 system()等调用会引发一些不可预知的错误，主程序中严禁调用 system()、popen()、fork()等系统函数。

提供一个简单的守护进程 system_call_daemon，专门来做 system 等 API 的处理，主程序与守护进程之间以进程间通信的方式进行交互。

相关代码位于 tools/libsystemcall 目录。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

4. 搭建 Camera 基础系统

4.1 系统分区编译

详细内容请参考【Ingenic Zeratul T23 SDK 使用说明】文档。

4.2 如何制作并进入 Normal u-boot

bootload 作为系统的引导程序，进行简单的硬件初始化工作。Zeratul 的 bootload 直接快速引导操作系统，因此不能编程，也没有 u-boot 的命令行功能。如果客户需要进入 u-boot 的命令行进行 debug 或者执行客户订制程序（如 TF 卡升级），可以在系统启动后，在命令行输入 recovery 进入 Normal u-boot。

u-boot 存在于 boot 分区中，开发者可以替换自己的 u-boot（第二阶段）。u-boot 的源码可以使用 os/uboot/u-boot/下的普通 u-boot 源码，编译出 u-boot 后，可以通过 build/pad_camera_u-boot.sh 将自己的 u-boot pad 到 boot.bin 中。使用方法：

```
usage:
./pad_camera_u-boot.sh -i <input_file> -o <output_file> -b <boot.bin fw>
eg:
./pad_camera_u-boot.sh -i u-boot-with-spl.bin -o boot_custom -
b ${ZRT_ENV_TOP_DIR}/firmware/camera/bootloader/boot.bin
```

isvp_t23n_msc0	T23N 卡起 uboot1
isvp_t23n_msc1	T23N 卡起 uboot2
isvp_t23n_sfcnand	T23N 可做 NAND u-boot
isvp_t23n_sfcnor	T23N 可做 Normal u-boot
isvp_t23zn_msc0	T23ZN 卡起 uboot1
isvp_t23zn_msc1	T23ZN 卡起 uboot2
isvp_t23zn_sfcnand	T23ZN 可做 NAND u-boot
isvp_t23zn_sfcnor	T23ZN 可做 Normal u-boot

4.3 调试阶段文件更新与固件烧录

4.3.1 文件更新

Camera 系统中，除了 config 分区为 rw 之外，system 为 ro，其它分区均为 ramfs 方式。调试时应用程序可以通过挂载 nfs 或者 tftp 的方式将应用程序 download 到/tmp/下执行

- 挂载 nfs 之前需要加载 nfs 依赖的 ko，可以通过执行 insmod_nfs 加载，再通

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

过 mount -o nolock <remote path> <local mount point>的方式挂载 nfs

- tftp 使用方法为：tftp -g -r rootfs_camera.cpio.lzo 192.168.1.100
- TF 的使用可以先通过 insmod_vfat 加载驱动，再通过 mount 方式挂载使用

如果需要更新处 config 分区外的文件系统中的文件，只能通过烧写完整的分区实现。例如需要更新主应用程序，主应用程序在 rootfs 分区中，那需要重新烧录 rootfs_camera.cpio.lzo 文件。

4.3.2 文件系统环境下烧录

大部分的烧录可以在进入 shell 后进行。方法为：

- 挂载 nfs 或者通过 tftp 将文件 download 到/tmp/下
- 烧写到分区 flashcp <src_file> <dest_partition>，其中分区为/dev/mtdX

例如：

```
mkdir -p /tmp/nfs
mount -o nolock 192.168.1.100:/nfsroot /tmp/nfs
flashcp /tmp/nfs/boot.bin /dev/mtd0
或者
flashcp /tmp/nfs/tag.bin /dev/mtd1
或者
flashcp /tmp/nfs/uImage.zrt /dev/mtd2
或者
flashcp /tmp/nfs/rootfs_camera.cpio.lzo /dev/mtd3
或者
flashcp /tmp/nfs/system.bin /dev/mtd5
sync
```

4.3.3 u-boot 环境下烧录

与普通 Linux 稍有不同，Zeratul Camera 的 bootloader 不能直接进入命令行，需要在进入系统 shell 后，输入 recovery 命令并回车，系统将重起进入 u-boot 的命令行。在 u-boot 命令行里可以通过 tftp 和 sf 命令来操作烧写 flash。这一部分的方法与普通 IPC 平台类似，命令与 TF 卡烧录一致，可参考《Ingenic_Zeratul_T23_烧录说明》。

4.3.4 TF 卡烧录

按住 bootsel 按键同时启动 Camera，会进入 TF 卡 boot，TF 启动卡以及烧录方法可参考《Ingenic_Zeratul_T23_烧录说明》。

4.3.5 烧录地址

首次烧录 camera 需要全擦 flash。后续更新时，如果不更新 config 分区（config 分区大小为 512KB），则不需要擦除最后的 512KB 空间。uImage.zrt 和 rootfs 的 offset 和 size 定义在 tag 分区的 bootinfo 中，此外，包括这两者的所有分区信息定义在
tag 分区的 cmdline 之中。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

4.4 恢复与升级

4.4.1 恢复初始状态与设置

恢复初始状态与设置，只需要把 config 分区的内容全部擦除，或者删除/config/.tag 文件，再重启即可。删除/config/.tag 文件后，下次重起时会重新格式化 config 分区。

4.4.2 升级

升级有两种方案：

- 直接升级方案
- recovery 升级

其中，**直接升级方案**的几个分区升级方法如下：

- boot：系统启动后 boot 分区不再进行访问，因此可以直接擦除再写入。注意：bootloader 如果损坏，设备将无法启动，只能进行售后修理（只能进行 TF 卡启动恢复，或者更换存储芯片）
- tag：tag 分区没有挂载，可以随时写入，但要注意擦写时需要与修改 tag 的动作做互斥（比如更新系统 env 参数）
- uImage：uImage 是在 bootloader 阶段加载的，在 uImage 运行起来之后，也不会访问 uImage 存储分区，因此可以随时擦除并写入升级后的数据
- rootfs：类似于 uImage，rootfs 可以随时擦除并写入新的数据
- recovery：系统正常启动不挂载 recovery 分区，因此 recovery 分区可以随时擦除并写入
- system：由于 system 分区存放系统的常用资源，所以再烧写之前需要 kill 掉 system 上执行的程序，close system 的文件句柄，并 umount system 分区，再进行擦除写入操作
- config：config 分区是 rw 分区，可以直接修改文件内容或添加删除文件。

recovery 升级方案升级方法：

概念介绍：

Recovery 系统：Recovery 系统是一个独立的小型操作系统，在一个 bin 中包含 uImage 以及 rootfs。即使正常系统的 rootfs 和 uImage 破坏，也可以进入 recovery 系统。

recovery 命令：系统中执行 recovery 命令，会重起并进入 recovery 系统

升级步骤：

将升级的 fw 下载到 TF 卡中，执行 recovery 命令，系统重起后进入 recovery 系统进行升级一个参考的例子是 tools/host/mk_update 目录下的 TF 卡 recovery 升级方式，TF 卡中存放 UPDATE.zip 升级包，进入 recovery 后会执行脚本，执行升级操作。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

对于直接升级方案和 recovery 升级方案，有两种升级包文件获取方法：

- TF 卡
- OTA（网络升级）

TF 卡挂载文件系统并获得升级包是最简单的升级方案，可以很简单的实现。OTA 方案更适合客户操作。往往需要将升级文件下载到系统上（tmpfs、TF 卡、或者 zramdisk 等），对于压缩包还需要解压。如果是直接升级方案，需要关闭一些功能模块，以释放足够的内存来存放升级包文件

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

5. MCU_WDT 功能

5.1 MCU_WDT 是什么功能？主要用来解决什么问题

MCU_WDT 是一个通过主控 SOC 和 MCU 之间定时通信，来实现主控 SOC 发生异常时可进行系统恢复的功能。通过 MCU_WDT，可以实现系统启动失败重启恢复，系统崩溃重启恢复。MCU_WDT 并不是 MCU 本身的 WDT，而是通过主控 SOC 和 MCU 之间的定时通信虚拟的 WDT 功能，即如果主控 SOC 没有按规定时间通信，则会触发 MCU_WDT 重启。

MCU_WDT 十分重要，实际产品上建议开启此功能。由于这个功能与 MCU 代码密切相关，所以提出概念，没有实际参考资料。

5.2 MCU_WDT 功能真值表

MCU_WDT	系统状态	WDT计时	说明
关	上电/断电	停止计时	功能关闭，任何状态下均没有WDT功能
开	上电	开启计时	功能打开，系统上电后MCU的WDT自动开始计时
开	断电	停止计时	功能打开，系统断电后MCU的WDT自动停止计时

5.3 MCU_WDT 的使用

- 喂狗方案：有 daemon 喂狗方案和用户程序喂狗的方案。
  - daemon 程序喂狗方案：不需要用户程序做除打开关闭 MCU_WDT 之外的操作，但这个方案并不能保护用户程序本身崩溃导致的异常。
  - 用户程序喂狗方案：用户程序中需要启动一个独立的线程来做喂狗

- 功能的打开和关闭
  - 功能打开：建议每次启动后都打开一次，MCU_WDT 可以做重复打开的操作
  - 功能关闭：正常情况下，用户程序不需要关闭 MCU_WDT，在 MCU 给主控 SOC 断电后，MCU 端会自动停止 WDT 的 Counter，直到下次上电才会再次自动打开。因此假设某次系统启动失败，MCU 的 Counter 到时后，依然会重新启动主控 SOC（若干次重启失败后会全部关闭系统电源保护电池，此情况下可能是发生了硬件异常，需要返修检查）。有如下情况可能需要关闭 MCU_WDT：USB 烧录
- OTA 及 recovery 时建议保持 MCU_WDT 功能

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

6. Camera 快速启动优化

Camera 需要快速启动，以第一时间抓拍到图像，并连接 WIFI。
快速启动优化主要有以下几个方向：

6.1 尽快运行主程序

主程序应该放置在 rootfs 中并第一时间加载运行。例如在 rcS 中，在环境变量设置完成之后即可运行主程序：

```
#!/bin/sh
#bootup_timer start >> /tmp/bootup_time
# Set Global Environment
export PATH=/system/bin:/bin:/sbin:/usr/bin:/usr/sbin
export LD_LIBRARY_PATH=/system/lib:/usr/lib
# Set transfer mode with MCU Environment (IIC or UART)
export TRANSFER_MODE=IIC
# Set Product mode with camera Environment (SUIT or SINGLE)
export PRODUCT_MODE=SINGLE
# Set callback script path Environment (default /usr/bin/shutdown.sh)
export CALLBACK_SCRIPT=
# Set Sensor Environment
export SENSOR=sc2235
# Set SOC Type Environment
export SOC_TYPE=SOC_T23Z
zrt_app & ####在设置完环境变量后即可运行主程序
insmod_wifi
if [ $? -ne 0 ]; then
echo "wifi driver load failed, reboot..."
reboot
fi
zrt_cam_daemon --sdcard_path=/tmp/mnt/sdcard &
```

切记：

主程序依赖的动态库要放在 rootfs 中，例如 libimp.so

主程序启动后首先初始化视频，之后可以并行处理其他的业务（比如 P2P、认证鉴权等等）

如果对某些配置文件有依赖，例如 config 分区下的配置文件，可以使用如下的方式探测等待

```
while (--timeout) {
        if (access(FILE_PATH, R_OK) == 0)
                break;
        usleep(100000);
}
if (timeout == 0)
printf("wait for XXX file timeout\n");
```


Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

6.2 缩短 rootfs 的加载时间

缩短加载时间的方式就是尽量优化 rootfs 的大小。有以下几个方面：

非启动必须的资源，尽量放置在 system 分区中，例如配网音频，字库文件等

主程序依赖的动态库，如果不是启动依赖，可以放到 system 分区中，但是要用过 dl_open 的方式加载。例如 libcrypto，这个库的大小比较大，只有个别时候才用得到，可以把要的接口简单封装，然后通过 dl_open 的方式动态调用这些接口

6.3 IRCUT 和 IRLED 的处理

由于 IR 会设置多种模式，且需要提前处理，并且需要用户程序配合，由于 T23 有 riscv 协助，在 uboot 期间就需要判断黑白夜视。大致黑白夜视流程如下：

1、由 cmdline 中 env 部分 ir_mode 参数来决定 ir 的切换方式，可动态修改。

2、uboot 期间根据 ir_mode 参数来决定切换方式，以及根据光敏值来判断当前白天夜视情况。具体切换动作在 uboot 中进行。

3、uboot 将白天夜视情况传递给 riscv，riscv 根据白天夜视情况选择不同的参数进行 2A 收敛。

4、riscv 最终会同步将白天夜视情况传递给内核，不支持 riscv 的 Sensor，由 uboot 直接传递白天夜视情况给内核。

5、内核阶段，会根据白天夜视情况来设置 ISP 的状态，以及恢复 IRCUT GPIO 的设置，防止 IRCUT 耗电以及烧毁。

6、内核期间会创建/proc/ir_status 接口给上层使用，以便上层得知内核当前的状态。

7、应用空间启动后，会读取/proc/ir_status 的状态，开机第一次是不会做任何切换，只是获取状态，待环境亮度发生变化的时候，上层根据实际的切换逻辑进行切换。

6.3.1 TAG ENV 处理方法

TAG 中存在一个 env 段，该段信息会在系统启动的时候 pad 到 cmdline 的后面，作为 cmdline 的一部分传递到内核，所以 TAG 中的 env 信息，其实是在整个系统运行过程中均可获得的。

提供一个工具 tag_env_info 用来读写 tag 分区中 env 的信息，使用方法：

./tag_env_info [操作] [标签] [参数] [值]

env 格式说明：

env 的格式遵循标准的 ini 格式，只是将换行由分号代替。

```
标准 ini 格式:
[token1]
param1=value1
param2=value2
[token2]
param3=value3
param4=value4
env 格式:
```


Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
[token1];param1=value1;param2=value2;[token2];param3=value3;param4=value4
```

IRCUT 设置方法：
标签为：IR
参数为：ir_mode
环境变量信息举例：[IR];ir_mode=auto;
值包含 3 种：
off： 常关（白天状态）
on： 常开（夜晚状态）
auto：根据光敏自动切换

```
示例:
./tag_env_info --set IR ir_mode auto
./tag_env_info --get IR ir_mode
```

注意：

执行过程中请避免断电

6.3.2 IRCUT IRLED GPIO 设置

IRCUT、IRLED 等设置在 tools/make_tag/sensor_start_ae_table/ 对应的 Sensor 文件中。如下图：

```
#include "libz_common.h"

int g_gpio_led_mode = LED_MODE_IGNORE;
int g_gpio_led = -1;
int g_gpio_ircut_mode = IRCUT_MODE_IGNORE;
int g_gpio_ircut_p = -1;
int g_gpio_ircut_n = -1;
int g_gpio_ircut_edge = 0;
int g_adc_value = -1;
int g_adc_direction = ADC_DIRECTION_0;
int g_adc_reference = -1;
int g_gpio_white_led = -1;
int g_gpio_white_led_status = -1;
```

如果不想让 uboot 控制任何数据，对应的参数都设置为-1 即可
具体参数含义如下图，对应的宏定义请参考 libz_common.h 文件说明

```
/**
 * IRLED
 * -1: 不受控制
 * 0: 高电平亮灯, 低电平灭灯
 * 1: 高电平灭灯, 低电平亮灯
 */
int gpio_led_mode;
int gpio_led;

/**
 * IRCUT
 * -1: 不受控制
 * 0: 电平触发方式 P 高电平 N 低电平 切夜视
 * 1: 电平触发方式 N 高电平 P 低电平 切夜视
 * 2: 沿触发方式 上升沿触发 切夜视
 * 3: 沿触发方式 下降沿触发 切夜视
 */
int gpio_ircut_mode;
int gpio_ircut_p;
int gpio_ircut_n;
int gpio_ircut_edge; /* 当触发方式为沿触发方式, 当前成员参数GPIO值生效 */

// ADC 阈值
int adc_value; /* 白天夜视切换阈值,该值小于0,不受控制 */
int adc_direction; /* 0: 小于阈值为夜视 1:大于阈值为夜视 */
```


Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

6.3.3 内核操作方法

内核处理代码位于 os/drivers/tx-isp-t23/tx-isp-fast.c，内核期间主要处理两件事情，一是防止 IRCUT 耗电以及烧毁，需要在内核期间恢复到 idle 状态，二是将创建 ir_status 节点，供上层应用获取内核处理状态。防止频繁切换。

```
// IRCUT 切回idle状态
ircut_gpio_init();
// 创建 ir_status 节点
proc_create("ir_status", S_IRUGO, NULL, &ir_status_proc_fops);
```

提供一个节点供上层读取相关信息 /proc/ir_status
该节点有 5 种状态，读取的值为字符串，分别为（常见 day、night 两种）：

ignore：未设置光敏范围和红外灯以及偏光片回调时，不处理，返回 ignore
off: 用戶层设置 IR mode 为常关（白天状态）
on: 用戶层设置 IR mode 为常开（夜晚状态）
day: 用戶层设置 IR mode 为自动切换，kernel 判断光敏为白天状态
night: 用戶层设置 IR mode 为自动切换，kernel 判断光敏为夜晚状态

6.4 ADC 取值和日夜切换阈值

在实际使用过程中，有可能会发现 uboot 中的 ADC 采样值和应用层通过 API 获取的采样值会存在差异，可能原因是由于参考电压取值不对导致。

T23 硬件设计默认的参考电压应该是 1.8V，最新的 uboot，我们提供接口可以设置对应的参考电压，这样示波器实际量到的电压和 ADC 取值应该是可用对应上的。

以 gc2063 为例，在 tools/make_tag/sensor_start_ae_table/libz_gc2063.c 文件中，我们提供了两个全局变量，如 g_adc_value：日夜切换阈值，g_adc_reference：参考电压，如果参考电压不设置，uboot 默认使用的是 1.8V。

```
#include "libz_common.h"

int g_gpio_led_mode = LED_MODE_IGNORE;
int g_gpio_led = -1;
int g_gpio_ircut_mode = IRCUT_MODE_IGNORE;
int g_gpio_ircut_p = -1;
int g_gpio_ircut_n = -1;
int g_gpio_ircut_edge = 0;
int g_adc_value = -1;
int g_adc_direction = ADC_DIRECTION_0;
int g_adc_reference = -1;

struct vol_start_value_table table_day[] = {
    {48,    20077},
    {49,    17998},
    {51,    15920},
    {53,    13842},
    {55,    11764},
```

日夜切换阈值
参考电压

应用程序 ADC 取值的参考电压由内核代码决定，具体的位置如下图所示：

```
#include <irq.h>

static unsigned int VREF_ADC = 1800;
#define AUXCONST   1024

#define ADC_ENABLE          0
#define ADC_DISABLE         1
#define ADC_SET_VREF        2

#ifndef BITS_H2L
#define BITS_H2L(msb, lsb)  ((0xFFFFFFFF >> (32-((msb)-(lsb)+1))) << (lsb))
#endif
-/work/zeratul/os/kernel/drivers/mfd/jz_adc_aux.c[POS=11,1][3%]16/10/20 - 12:06
```


Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

另外关于 ADC 日夜切换阈值，提供了一个可动态配置的方式，需要在 cmdline 的 env 中进行设置 adc_value 对应数值。如果该参数设置，则使用该参数值，如果该参数未设置，则使用 libz_gc2063.c 中对应的阈值。

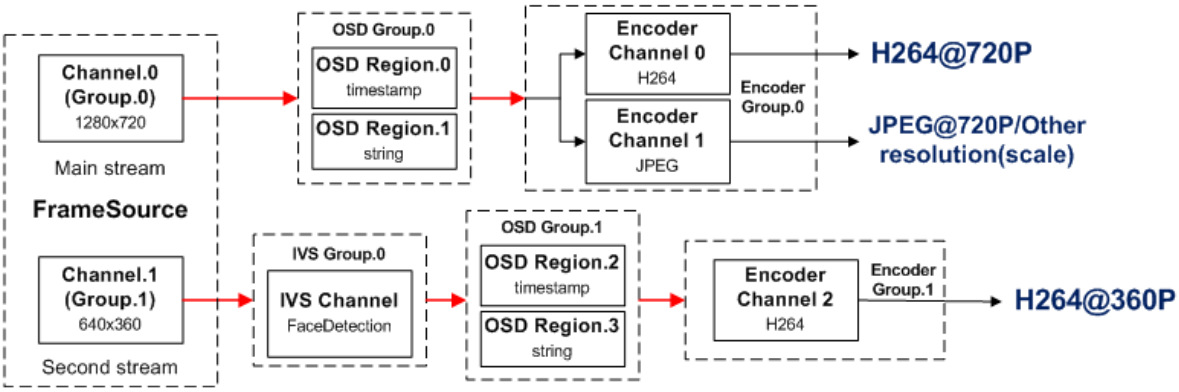
该参数可由 tag_env_info 来动态设置。

```
if [ ${ZRT_ENV_SOC_TYPE} = SOC_T23Z ]; then
if [ ${ZRT_ENV_KERNEL_RELEASE} = 4.4.94-Archon ]; then
    echo "SOC_T23Z KERNEL VERSION 4.49"
    CMDLINE="console=${UART},115200n8 mem=80M@0x0 rmem=48M@0x5000000 root=/dev/ram0 rw rdinit=/linuxrc mtdparts=jz_sfc:256K(boot),352K(tag),6M(kernel),3M
    (rootfs),3M(recovery),2976K(system),512k(config),16M@0(all) lpj=6955008 ${QUIET}"
    BOOTINFO="kernel=6M@0x98000 rootfs=3M@0x698000"
elif [ ${ZRT_ENV_KERNEL_RELEASE} = 3.10.14 ]; then
    CMDLINE="console=${UART},115200n8 mem=80M@0x0 rmem=48M@0x5000000 root=/dev/ram0 rw rdinit=/linuxrc mtdparts=jz_sfc:256K(boot),352K(tag),5M(kernel),3M
    (rootfs),2560K(recovery),4512K(system),512k(config),16M@0(all) lpj=6955008 ${QUIET}"
    BOOTINFO="kernel=5M@0x98000 rootfs=3M@0x598000"
fi
    ENVINFO="senv;[HW];init_vw=1920;init_vh=1080;nrvbs=3;mode=0;adc_value=100;eenv;"
    SOC_TYPE=t23
    RISCV_FW="../riscv_fw/riscv_fw_${ZRT_ENV_SENSOR}.bin"
fi
```

6.5 NRVBS 说明

在快起配置的时候，会有一个 NRVBS 参数设置，这个实际上是配置内核缓存 buffer 的个数，且应用层会共享这个 buffer，内核的配置由 cmdline 中 env 来决定，所以应用程序中配置的 nrvbs 参数需要和 cmdline 中一致（针对主码流），否则会出现异常问题。

nrvbs 具体的作用是什么呢？我们以双路码流结构举例，如下：



Framesource 负责获取 YUV 原始图像数据，OSD 负责在 YUV 之上添加 logo 时间戳等图像信息，Encoder 负责将 YUV 数据编码成 H264、H265 数据。对应的次码流中还有 IVS 模块，IVS 则是对 YUV 数据做图像检测类算法。

YUV 数据会在各个模块中传递和使用，为了提高传递效率，各个模块之间使用的是对应 YUV buffer 的指针数据，但是各个模块之间的处理顺序是有要求的。

以一个 YUV buffer 为例，那么 framesource 获取到图像数据后，传递给 OSD，OSD 处理完成之后再传递给 Encoder，当 Encoder 处理完成之后，最后再传递给 framesource 获取下一帧图像，那么这个流程就是一个串行执行的流程。

我们配置的 nrvbs 即 framesource 中使用的 YUV buffer 的个数。

个数越多，并行的效率就越高，比如 nrvbs 配置为 3，framesource 再处理第 3 个

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

buffer 的时候，OSD 在处理第 2 个 buffer，Encoder 在处理第 1 个 buffer，这样 3 个硬件单元同时工作处理不同的 buffer，提高并行效率。

在某些场景下，如果出现帧率不足的情况，有可能就是 nrvbs buffer 个数太少导致的，可用适当提高个数试试。推荐使用 2，在内存充足的情况下面，建议使用 3、4。

6.6 快速启动分辨率和 nrvbs 设置方法

由于上层与驱动层需要密切配合，所以，有些配置信息需要提前配置，如 sensor 的宽高信息和所配置的 video buffer 个数。

这些信息将被写入到 TAG 分区的环境变量中。并且这些信息需要和主程序的配置完全一致。可以有两种方式写入：

a）、在设备串口里，利用 tag_env_info 工具写入

```
tag_env_info --set HW init_vw 1920 // sensor 的宽
tag_env_info --set HW init_vh 11080 // sensor 的高
tag_env_info --set HW nrvbs 2 // video buffer 个数，默认设置为 2，内存够用的话，建议设置为 3 到 4 个
```

b）、在制作 TAG 的时候，用 tag_generator 工具直接设置

```
./tag_generator_host --create --tag_path=tag.bin --cmdline="console=ttyS1, 115200n8 mem=80M@0x0 rmem=48M@0x5000000 root=/dev/ram0 rw rdinit=/linuxrc mtdparts=jz_sfc:256K(boot),352K(tag),5M(kernel),3M(rootfs),2560K(recovery),4512K(system),512k(config),16M@0(all) lpj=6955008 quiet"
--env="senv;[HW];init_vw=1920;init_vh=1080;nrvbs=2;eenv;"
```

6.7 帧率的修改

由于 T23 中有 riscv，riscv 推荐使用高帧率，但是实际产品帧率通常为 15fps，那实际的降帧是怎么操作的呢？

Riscv 中的帧率由 Sensor 初始化序列决定，也就是 tools/make_tag/sensor_init_setting 文件夹中对应的 Sensor 初始化序列决定。

在内核期间，ISP 驱动中，我们提供了一些回调以及 API，如 tx_isp_frame_done_int_hander 回调函数，该回调函数是硬件产生 frame done 中断之后，每产生一个中断，则会回调该函数，count 为 1 开始计数。

我们可以在该回调函数中来切换帧率，由于 Sensor 一般配置都是隔帧生效，所以通常在第一帧的时候设置帧率，实际在第 3 帧的时候生效。这样做的目的是前两帧还是高帧率出图，第 3 帧切到 15fps，如果存在前两帧图像效果较差的话，我们可以将其丢弃，从而提高第一帧图像出图速度。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
static int tx_isp_frame_done_int_handler(int count)
{
    if(count == 1) {
        tx_isp_tuning_set_fps(15, 1); /* 最终实现在中断下文中实现 */
    }

    return 0;
}

struct tx_isp_callback_ops g_tx_isp_callback_ops = {
    .tx_isp_riscv_hf_prepare = tx_isp_riscv_hf_prepare,
    .tx_isp_riscv_hf_resize = tx_isp_riscv_hf_resize,
    .tx_isp_frame_done_int_handler = tx_isp_frame_done_int_handler,
};
~/work/zeratul/os/drivers/tx-isp-t31/tx-isp-fast.c[POS=456,5][96%]31/08/20 - 21:04
```

6.8 系统内存以及优化

6.8.1 系统内存说明

芯片真实的物理内存会被划分为两块，由 cmdline 来划分，如：mem=80M@0x0 rmem=48M@0x5000000，其中 mem 用于 linux 系统内存，rmem 用于媒体内存。

具体媒体内存如何使用，请参考《Ingenic_Zeratul_T23_媒体内存池使用说明》文档。

如下图，在内核启动打印中，会存在如图所示的信息。包括以下 4 个信息：

```
mem=80M@0x0 rmem=48M@0x5000000
rd_start=0x80600000 rd_size=0x4a2600
Memory:69860K/81920K
3175K kernel code,12060k reserved,736k data,184k init
```

```
[    0.000000] Movable zone start for each node
[    0.000000] Early memory node ranges
[    0.000000]   node   0: [mem 0x00000000-0x04fffffff]
[    0.000000] On node 0 totalpages: 20480
[    0.000000] free_area_init_node: node 0, pgdat 803e16e0, node_mem_map 81000000
[    0.000000]   Normal zone: 160 pages used for memmap
[    0.000000]   Normal zone: 0 pages reserved
[    0.000000]   Normal zone: 20480 pages, LIFO batch:3
[    0.000000] Primary instruction cache 32kB, 8-way, VIPT, linesize 32 bytes.
[    0.000000] Primary data cache 32kB, 8-way, VIPT, no aliases, linesize 32 bytes
[    0.000000] pls check processor_id[0x00d00100],sc_jz not support!
[    0.000000] MIPS secondary cache 128kB, 8-way, linesize 32 bytes.
[    0.000000] pcpu-alloc: s0 r0 d32768 u32768 alloc=1*32768
[    0.000000] pcpu-alloc: [0] 0
[    0.000000] Built 1 zonelists in Zone order, mobility grouping off.  Total pages: 20320
[    0.000000] Kernel command line: console=ttyS0,115200n8 mem=80M@0x0 rmem=48M@0x5000000 root=/dev/ram0 rw rdinit=/linuxrc mtdparts=jz_sfc:256K(boot),352K(tag),5M(kernel),3M(rootfs),256
0K(recovery),4512K(system),512k(config),16M@0(all) lpj=6955008 quiet senv;[HW];init_vw=2500;init_vh=1440;nrvbs=2;mode=0;eenv; lzo_size=2140650 rd_start=0x80600000 rd_size=0x4a2600
[    0.000000] PID hash table entries: 512 (order: -1, 2048 bytes)
[    0.000000] Dentry cache hash table entries: 16384 (order: 4, 65536 bytes)
[    0.000000] Inode-cache hash table entries: 8192 (order: 3, 32768 bytes)
[    0.000000] Memory: 69860k/81920k available (3175k kernel code, 12060k reserved, 736k data, 184k init, 0k highmem)
[    0.000000] SLUB: HWalign=32, Order=0-3, MinObjects=0, CPUs=1, Nodes=1
[    0.000000] Preemptible hierarchical RCU implementation.
[    0.000000] NR_IRQS:358
[    0.000000] clockevents_config_and_register success.
[    0.000013] Calibrating delay loop (skipped) preset value.. 1391.00 BogoMIPS (lpj=6955008)
[    0.000024] pid_max: default: 32768 minimum: 301
[    0.000186] Mount-cache hash table entries: 512
[    0.000602] Initializing cgroup subsys debug
[    0.000622] Initializing cgroup subsys freezer
[    0.002183] NET: Registered protocol family 16
[    0.002490] register process exit failed.
[    0.009068] bio: create slab <bio-0> at 0
[    0.010472] jz-dma jz-dma: JZ SoC DMA initialized
[    0.010695] i2c-gpio i2c-gpio.1: using pins 57 (SDA) and 58 (SCL)
[    0.010760] i2c-gpio i2c-gpio.2: using pins 6 (SDA) and 7 (SCL)
[    0.012067] Switching to clocksource jz_clocksource
[    0.012508] NET: Registered protocol family 2
[    0.013844] TCP established hash table entries: 1024 (order: 1, 8192 bytes)
```

`mem=80M@0x0 rmem=48M@0x5000000`：

该信息主要是来划分系统内存和媒体内存。格式为：大小@地址，所以我们平时在修改内存大小的时候，只改了前面大小，却忘记修改后面地址偏移了，会导致启动出现各种异常现象，这里需要注意。

`rd_start=0x80600000 rd_size=0x4a2600`：

由于 rootfs 是以 ramfs 的方式存在，所以需要在 ram 上申请一段内存来存放 rootfs，上述信息就是 rootfs 存放的位置和大小。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

`Memory:69860K/81920K`：

这个是系统内存的情况，81920K 实际是系统内存分配大小，由上面的 80M@0x0 决定，69860K 是系统可用的实际内存，那 81920-69860=12060K 这个内存去哪里了呢，我们可以看下面的内存信息，会发现有一个 12060K reserved 信息，那么这个内存其实是预留给内核和 rootfs 了。

`3175K kernel code,12060k reserved,736k data,184k init`：

从这个信息可用看到，12060K 的预留内存，一部分是给内核使用，包括内核代码段、数据段、以及 init 段。剩余的 12060 - 3175 - 736 - 184 = 7965K，这个其实会给 ramfs 使用，以及其他资源使用。

所以 128M 实际物理内存被划分如下：

系统可用内存 69860K	系统预留内存 12060K 包含内核、ramfs	媒体内存 48M

6.8.2 系统内存优化

通过上面分析，如果想优化系统内存，主要的一个思路就是，减小系统预留内存，和优化媒体内存。

1、减小系统预留内存

可用适当优化内核，内核不必要的选项去掉，目前推荐的是将内核的符号表去除，其余部分优化空间不大，可不做优化，如下箭头位置去掉：

```
--- Configure standard kernel features (expert users)
[*]   Sysctl syscall support
[*]   Load all symbols for debugging/ksymoops
[ ]     Include all symbols in kallsyms
[*]   Enable support for printk
[*]   BUG() support
[*]   Enable ELF core dumps
[*]   Enable full-sized data structures for core
[*]   Enable futex support
[*]   Enable eventpoll support
[*]   Enable signalfd() system call
[*]   Enable timerfd() system call
[*]   Enable eventfd() system call
[*]   Use full shmem filesystem
[*]   Enable AIO support
```

其次就是减少 rootfs 的大小，建议 rootfs 中只存放对快起有要求的必要程序和库文件，如果对快起没有要求的程序或库文件建议放到 system 分区。

2、优化媒体内存

媒体内存我们有提供计算器根据实际需求计算，请参考如下文档：《rmem_calculator_T23.xlsx》

也可用通过另外一种方式来处理，先将 rmem 配置较大，可以正常跑起系统，

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

码流信息也按照实际使用的最大场景配置，当系统正常运行时，通过 cat /tmp/continuous_mem_info 来得到实际的使用情况。

```
Mem Total Size: 50331648
Mem Used  Size: 31825920
Mem Free  Size: 18505728
Mem Usage Rate: 63%
```

然后再将实际的使用大小配置给 rmem 即可。

另外，建议如果系统内存不够的情况，nrvbs 建议配置为 2。

6.9 分区信息如何调整

正常快起的分区信息一般由 cmdline 来决定，recovery 系统的分区一般由 pad 的 uboot 来决定。

默认 uboot 分区大小为 256K，如果 uboot 分区大小需要调整，则需要原厂重新提供调整过分区的 SPL 固件。建议 uboot 分区尽量不要做调整。

6.9.1 正常系统下分区调整

1、正常系统的 cmdline 由 tag 决定，所以需要修改 tag 制作脚本，tools/make_tag/tag_generator/build_tag_t23.sh

2、修改./build/build_camera_all.sh，这个脚本会最终将各个分区打包到一个固件中，所以该分区信息也需要对应修改。

6.9.2 Recovery 系统下的分区调整

Recovery 系统下的分区由 pad 的 uboot 代码决定，os/uboot/u-boot/include/configs/isvp_t23.h 文件中。修改如下信息即可：

```
336
337 #ifdef CONFIG_SPL_MMC_SUPPORT
338     #define CONFIG_BOOTARGS BOOTARGS_COMMON " init=/linuxrc root=/dev/mmcblk0p2 rw rootdelay=1"
339 #elif defined(CONFIG_SFC_NOR)
340     #define CONFIG_BOOTARGS BOOTARGS_COMMON " init=/linuxrc rootfstype=squashfs root=/dev/mtdblock2 rw mtdparts=jz_sfc:256k(boot),2560k(kernel),2048k(root),-(appfs)"
341 #elif defined(CONFIG_SFC_NAND)
342     #define CONFIG_BOOTARGS BOOTARGS_COMMON " ip=off init=/linuxrc ubi.mtd=2 root=ubi0:rootfs ubi.mtd=3 rootfstype=ubifs rw"
343 #endif
344
345 /**
346  * Boot command definitions.
347  */
348 #define CONFIG_BOOTDELAY 1
349
350 #ifdef CONFIG_SPL_MMC_SUPPORT
351 #define CONFIG_BOOTCOMMAND "mmc read 0x80600000 0x1800 0x3000; bootm 0x80600000"
352 #endif
353
354 #ifdef CONFIG_SFC_NOR
355     #define CONFIG_BOOTCOMMAND "sf probe;sf read 0x80600000 0x40000 0x280000; bootm 0x80600000"
356 #endif /* CONFIG_SFC_NOR */
357
358 #ifdef CONFIG_SFC_NAND
359     #define CONFIG_BOOTCOMMAND "sfcnand read 0x100000 0x400000 0x80600000 ;bootm 0x80600000"
360 #endif
361
362 /**
363  * Drivers configuration.
```

分区信息

启动命令

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

6.10 图像尺寸缩放说明

6.10.1 图像缩放配置

对于 T23 而言，图像有缩放需求，缩小没有问题，这里主要说明一下放大操作。以 200 万像素放大到 300 万像素为例。

由于内核我们会提前缓存数据，内核配置如下，需要修改

```
os/kernel/arch/mips/xburst/lib/isp-t23/tx-isp-fast.c
```

然后重新编译内核即可。

注意：如需保持市场角缩放，crop 参数保持和 sensor 原始尺寸一致。

```
format.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
format.pix.field = V4L2_FIELD_ANY;
if(frame_channel_width) {
    // format.pix.width = frame_channel_width;
    // format.crop_width = frame_channel_width;
    // format.scaler_out_width = frame_channel_width;
    format.pix.width = 2048;
    format.crop_width = 1920;
    format.scaler_out_width = 2048;
} else {
    format.pix.width = FRAME_CHANNEL_DEF_WIDTH;
    format.crop_width = FRAME_CHANNEL_DEF_WIDTH;
    format.scaler_out_width = FRAME_CHANNEL_DEF_WIDTH;
}

if(frame_channel_height) {
    // format.pix.height = frame_channel_height;
    // format.crop_height = frame_channel_height;
    // format.scaler_out_height = frame_channel_height;
    format.pix.height = 1536;
    format.crop_height = 1080;
    format.scaler_out_height = 1536;        You, 1秒钟前 · Uncommitted changes
} else {
    format.pix.height = FRAME_CHANNEL_DEF_HEIGHT;
    format.crop_height = FRAME_CHANNEL_DEF_HEIGHT;
    format.scaler_out_height = FRAME_CHANNEL_DEF_HEIGHT;
}
```

缩放参数写死到300M分辨率，裁剪参数保持sencor原尺寸

对于应用层来说，以 sample 为例，在 sample-common.c 的

```
struct chn_conf chn[FS_CHN_NUM]
```

中修改如下 scaler 参数，将分辨率调整到需要的分辨率即可。

```
        // .scaler.enable = 0,
        .scaler.enable = 1,

        // .picWidth = SENSOR_WIDTH,
        // .picHeight = SENSOR_HEIGHT,
        .picWidth = 2048,
        .picHeight = 1536,
    },
    .framesource_chn =    { DEV_ID_FS, CH0_INDEX, 0},
    .imp_encoder = { DEV_ID_ENC, CH0_INDEX, 0},
},
```

缩放参数打开，调整目标分辨率

另外，cmdline 的 env 中，会有图像的宽高信息，这些信息需要配置给内核，以及

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

SDK 使用，所以对应的 cmdline 也需要修改成对应的分辨率，如下：

修改 tools/make_tag/tag_generator/build_tag_t23.sh 文件

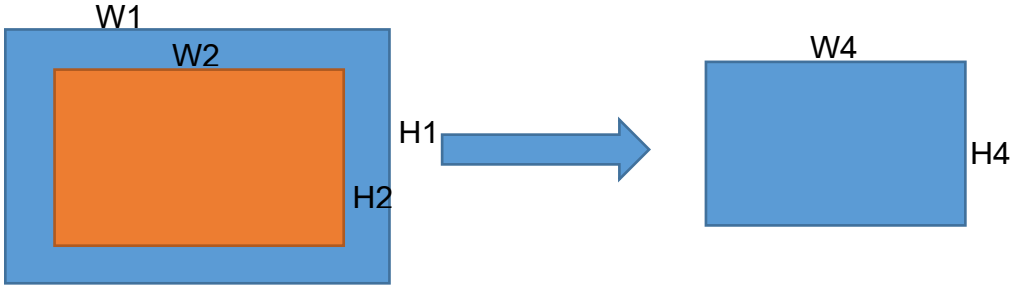
```
if [ ${ZRT_ENV_SOC_TYPE} = SOC_T23Z ]; then
    CMDLINE="console=${UART},115200n8 mem=36M@0x0 rmem=28M@0x2400000 root=/dev/ram0 rw rdinit=/linuxrc mtdparts=jz_sfc:256K(boot),352K
    (tag),5M(kernel),3M(rootfs),2560K(recovery),4512K(system),512k(config),16M@0(all) lpj=6955008 ${QUIET}"
    BOOTINFO="kernel=5M@0x98000 rootfs=3M@0x598000"
fi
    ENVINFO="senv;[HW] init_vw=2048;init_vh=1536;nrvbs=3;mode=0;pwm_duty=20;pwm_freq=80000;LED_GPIO_49=1;eenv;"
    SOC_TYPE=t23
    RISCV_FW="../riscv_fw/riscv_fw_${ZRT_ENV_SENSOR}.bin"
```

6.10.2 图像缩放裁剪计算方法

由于 T23 硬件要求，需要先缩放，再裁剪。在计算缩放裁剪尺寸的时候会比较绕，请套用下面公式即可。

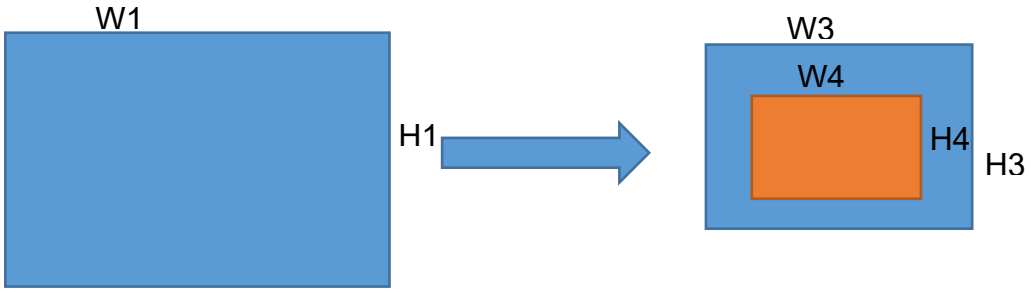
1、需求

如下图，W1* H1 为 Sensor 原始尺寸，W2*H2 为我们需要裁剪掉的部分，W4*H4 是我们目标尺寸。



2、实际操作

实际操作需要先缩放，再裁剪，其中 W3*H3 是缩放后的尺寸，W4*H4 是我们的目标尺寸，如下图。



3、计算

已知变量：W1、H1、W2、H2、W4、H4　求：W3、H3

由于等比例

W1/W3 = W2/W4

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

H1/H3 = H2/H4
得到
W3 = W1/(W2/W4)
H3 = H1/(H2/H4)

假设：
Sensor 分辨率为 1080P，需要裁剪分辨率为 720P，最终缩放输出分辨率为 VGA。
W3 = 1920 / (1280/640) = 960
H3 = 1080 / (720/480) = 720
所以缩放配置参数应该为：

```
.scaler.enable = 1,
.scaler.outwidth = 960,
.scaler.outheight = 720,

.crop.enable = 1,
.crop.left = (960 - 640) / 2, // 裁剪顶点坐标，可按实际配置
.crop.top = (720 - 480) / 2, // 裁剪顶点坐标，可按实际配置
.crop.width = 640,
.crop.height = 480,

.picWidth = 640,
.picHeight = 480,
```

6.11 ISP 驱动介绍

ISP 驱动代码路径位于 os/drivers/tx-isp-t23 文件夹中。主要由 tx-isp-fast.c 文件代码来实现。

6.11.1 回调函数

主要提供以下 3 个回调函数，具体的实现均在以下回调函数中实现。

```
struct tx_isp_callback_ops g_tx_isp_callback_ops = {
    .tx_isp_riscv_hf_prepare = tx_isp_riscv_hf_prepare,
    .tx_isp_riscv_hf_resize = tx_isp_riscv_hf_resize,
    .tx_isp_frame_done_int_handler = tx_isp_frame_done_int_handler,
};
```

tx_isp_riscv_hf_prepare：
主要用来与 RISC-V 小核同步信息，获取小核的统计结果，并且发送命令停止小核工作。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

tx_isp_riscv_hf_resize：

ISP 驱动流程，主要实现 Sensor 配置，ISP 配置，以及 framesource 的 buffer 和分辨率的设置，并启动内核出图。

tx_isp_frame_done_int_handler：

提供帧中断回调，当 T23 接收到一帧图像数据后，则会触发一次中断回调。目前在该函数中主要实现帧率切换动作。

注意：该函数为中断回调函数，不能在该函数中实现耗时操作，和调度操作。

6.11.2 分辨率和 buffer 配置

在 frame_channel_fast_start 函数中，有以下代码，主要用来配置 ISP 主通道的缩放裁剪和最终分辨率参数。默认情况下面不开启裁剪，只开启缩放功能。如果对分辨率裁剪有要求的话，需要手动修改以下配置。按照上文中提到的计算方式计算。

```
format.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
format.pix.field = V4L2_FIELD_ANY;
if(frame_channel_width) {
    format.pix.width = frame_channel_width;
    format.crop_width = frame_channel_width;
    format.scaler_out_width = frame_channel_width;
} else {
    format.pix.width = FRAME_CHANNEL_DEF_WIDTH;
    format.crop_width = FRAME_CHANNEL_DEF_WIDTH;
    format.scaler_out_width = FRAME_CHANNEL_DEF_WIDTH;
}

if(frame_channel_height) {
    format.pix.height = frame_channel_height;
    format.crop_height = frame_channel_height;
    format.scaler_out_height = frame_channel_height;
} else {
    format.pix.height = FRAME_CHANNEL_DEF_HEIGHT;
    format.crop_height = FRAME_CHANNEL_DEF_HEIGHT;
    format.scaler_out_height = FRAME_CHANNEL_DEF_HEIGHT;
}
format.pix.pixelformat = V4L2_PIX_FMT_NV12;
format.crop_enable = 0;
format.crop_top = 0;
format.crop_left = 0;
format.scaler_enable = 1;
format.pix.colorspace = V4L2_COLORSPACE_SRGB;
format.rate_bits = 0;
format.rate_mask = 1;
```

以下代码是用来配置 buffer 参数，根据 cmdline 中传入的 nrvbs 来决定。会自动配置，一般不需要修改。分辨率和 buffer 配置完成后，就可以启动出图，内核开始缓存图片数据。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
if(frame_channel_nrvbs) {
    count = frame_channel_nrvbs; /*nrVBs*/
} else {
    count = TX_ISP_NRVBS; /*nrVBs*/
}
ret = frame_channel_set_channel_banks(chan, (unsigned long)&count, K_MODE);
if(ret != 0) {
    printk(KERN_ERR "frame_channel_set_channel_banks failed\n");
}


for(buf_i = 0; buf_i < count; buf_i++) {
    struct v4l2_buffer buf;
    memset(&buf, 0, sizeof(struct v4l2_buffer));

    buf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
    buf.memory = V4L2_MEMORY_USERPTR;
    buf.index = buf_i;
    buf.m.userptr = addr + buf_i * (frame_channel_width * ((frame_channel_height + 0xF) & ~0xF) * 3 / 2);
    buf.length = (frame_channel_width * ((frame_channel_height + 0xF) & ~0xF) * 3 / 2);

    ret = frame_channel_vb2_qbuf(chan, (unsigned long)&buf, K_MODE);
    if(ret != 0) {
        printk(KERN_ERR "frame_channel_vb2_qbuf failed\n");
    }
}


type = V4L2_BUF_TYPE_VIDEO_CAPTURE;

// 启动出流
ret = frame_channel_vb2_streamon(chan, (unsigned long)&type, K_MODE);
if(ret != 0) {
    printk(KERN_ERR "frame_channel_vb2_streamon failed\n");
}
```

6.11.3 API 说明

ISP 驱动中可调用的 API 主要位于以下两个文件中。
os/drivers/tx-isp-t23/include/tx-isp-device.h
os/drivers/tx-isp-t23/include/tx-isp-tuning.h

tx-isp-device.h 中主要提供与 risc-v 交互的 API，以及 Sensor、ISP 初始化配置相关的 API，具体细节请打开该文件查阅。

tx-isp-tuning.h 中主要提供与 ISP 效果参数相关的 API，比如：图像亮度、对比度、饱和度、锐度、图像翻转、最大 again、最大曝光等参数的设定，具体细节请打开该文件查阅。

6.11.4 编译

ISP 驱动编译结构默认是做好的，只需要执行：

```
make clean && make && make release
```

即可将 ISP 驱动编译并拷贝到内核中。然后再进入到内核中重新编译内核即可。

6.11.5 丢帧和 debug

为了获得更好的效果，内核提供了一个丢帧接口，由于 3D 降噪功能是需要参考帧，第一帧数据主要用来统计，到第二帧才能真正介入，所以在低照度情况下，第一帧噪声会比较多，所以至少需要丢一帧，调用以下 API 进行丢帧即可。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
/**
 * @fn int tx_isp_discard_frame(uint32_t frame);
 *
 * 内核丢帧接口
 *
 * @param [frame] :需要丢弃的帧格式
 *
 * @retval 0 成功
 * @retval 非0 失败，返回错误码
 *
 * @remark 最大可控帧数为16帧，支持任意丢帧，frame 填充方式，对应 bit 为 1 则不丢弃，对应bit为0，则丢弃。
 * 如 丢弃前面两帧 frame = 0xfffffffc，如果想丢弃 第一帧和第三帧，frame = 0xfffffffa
 *
 * @attention 该函数建议在tx_isp_riscv_hf_prepare函数中调用或在stream on之前调用即可
 */
int tx_isp_discard_frame(uint32_t frame);
```

在图像出现异常的时候，或者需要进行起始 AE 参数表格统计的时候，可以使能以下 debug 接口，使能之后，内核会将对应的 EV 参数打印出来，可供分析和记录。

```
/**
 * @fn int tx_isp_enable_debug(int en, int count);
 *
 * 是否开启debug模式
 *
 * @param [in] en 0:关闭debug  1: 开启debug
 * @param [in] count :需要打印的帧数
 *
 * @retval 0 成功
 * @retval 非0 失败，返回错误码
 *
 * @remark 开启debug模式后，每帧的AE、AWB信息将会通过dmesg打印出来，方便调试
 *
 * @attention 该函数可在任意位置调用
 */
int tx_isp_enable_debug(int en, int count);
```

6.11.6 帧率切换

由于 risc-v 中默认是以较高帧率跑，到内核期间，帧率需要恢复成产品需要的帧率，一般低功耗产品所需的帧率为 15fps，所以内核期间默认会切到 15fps。通常我们会在中断回调函数中去切换帧率。如下图片所示：

```
static int tx_isp_frame_done_int_handler(int count)
{
    if(count == 1) {
        tx_isp_tuning_set_fps(15, 1); /* 最终实现在中断下文中实现 */
    }

    return 0;
}
```

为什么要在回调函数中进行帧率切换？

通常我们内核会配置丢帧，一般默认是丢弃两帧，为了提高出图速度，所以丢弃的帧我们尽量使用高帧率的。另外，一般 Sensor 的配置都是隔帧生效，所以我们在第一帧开始切帧率，实际要到第三帧才真正切到 15fps，所以前面两帧还是以高帧率跑的，这样丢帧和帧率切换配合就可以优化一定的出图时间。

6.11.7 出图时间优化

内核时间打印可以通过 DEBUG_TTFF();来打印对应的时间，该时间从上电开始计时。

部分 sensor 由于帧率达不到很高，risc-v 中统计的帧数不是很多，在内核默认是

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

等 6 帧，有可能会出现等的时间较长，这个时候，可以将以下图片中的 frame_count 参数改小一点，从而优化出图时间。

```
static int tx_isp_riscv_hf_prepare(void)
{
    int ret = 0;
    int timeout = 200;
    uint32_t frame_count = 0;

    DEBUG_TTFF("tx_isp_stop_riscv start");

    ret = tx_isp_riscv_is_run();
    if (ret < 0) {
        riscv_is_run = 0;
    } else {

#if 1
        timeout = 1000;
        while(timeout--) {
            ret = tx_isp_get_riscv_framecount(&frame_count);
            if(ret < 0) {
                printk("Error: %s tx_isp_get_riscv_framecount failed ret:%d\n", __func__, ret);
                return ret;
            }

            if((frame_count >= 6) && (frame_count != -1))
                break;

            mdelay(1);
        }

        DEBUG_TTFF("wait frame done");
#endif

        ret = tx_isp_stop_riscv();
        if(ret < 0) {
            printk("Error: %s tx_isp_stop_riscv failed ret:%d\n", __func__, ret);
            return ret;
        }
```

内核默认丢帧参数配置的是两帧，要求是至少丢一帧，所以在优化出图时间的时候，可以将丢帧参数改为一帧，只要图像没有明显异常即可。这样可以优化一帧的时间。另外，上面提到的帧率切换，帧率切换配合丢帧来操作，丢弃的帧跑较高帧率，这样可以节省一定时间。

6.11.8 日夜切换

日夜判断在 uboot 中判断，同时 uboot 会切换 IRCUT，由于 IRCUT 切换需要耗时，所以一般是 uboot 中触发切换，内核期间恢复 IRCUT 电平，如果不恢复，长时间后有可能会导致 IRCUT 一直供电从而功耗损失，且有烧毁 IRCUT 的风险。

另外内核期间 ISP 被重新初始化，所以需要重新设置日夜模式，参考如下图所示。这些接口可以在 tx_isp_riscv_hf_resize 函数中实现，放到 tx_isp_video_s_stream 之前调用即可。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
// 切换白天夜视模式
if(g_is_night == IR_STATUS_NIGHT) {
    isp_core_tuning_switch_day_or_night(0);
}

// IRCUT 切回idle状态
ircut_gpio_init();
// 创建 ir_status 节点
proc_create("ir_status", S_IRUGO, NULL, &ir_status_proc_fops);

// Stream on
ret = tx_isp_video_s_stream(module, 1);
if(ret < 0) {
    printk("tx_isp_video_s_stream failed\n");
    return ret;
}
```

注意：有些 IRCUT 芯片需要比较长的时间才能切过来，IRCUT 如果在这个位置恢复还达不到切换条件，建议可以放到 tx_isp_frame_done_int_handler 函数中进行恢复。count 可以根据实际情况配置。

```
static int tx_isp_frame_done_int_handler(int count)
{
    if(count == 1) {
        tx_isp_tuning_set_fps(15, 1); /* 最终实现在中断下文中实现 */
    }

    if(count == 5) {
        // IRCUT 切回idle状态
        ircut_gpio_init();
    }

    return 0;
}
```

6.11.9 图像镜像翻转

ISP 驱动中支持图像镜像翻转功能，调用以下 API 即可，在第一帧图像即可生效。如果产品支持正装和倒装场景，那么这个功能是需要动态设置的，建议把用户配置写入到 cmdline 的 env 段，在内核起来的时候去读取该参数，然后通过以下接口设置即可。

该接口可以在 tx_isp_riscv_hf_resize 函数中实现，放到 tx_isp_video_s_stream 之前调用即可。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
/**
 * ISP HV Flip模式。
 */
typedef enum {
    ISP_CORE_FLIP_NORMAL_MODE = 0,  /**< 正常模式 */
    ISP_CORE_FLIP_H_MODE = 1,       /**< 镜像模式 */
    ISP_CORE_FLIP_V_MODE = 2,       /**< 翻转模式 */
    ISP_CORE_FLIP_HV_MODE = 3,  /**< 镜像并翻转模式 */
    ISP_CORE_FLIP_MODE_BUTT,
} ISP_CORE_HVFLIP;

/**
 * @fn int tx_isp_tuning_set_hv_flip(ISPHVFLIP hvflip)
 *
 * 设置HV Flip的模式.
 *
 * @param[in] hvflip HV Flip模式.
 *
 * @retval 0 成功
 * @retval 非0 失败，返回错误码
 *
 * @attention 在使用这个函数之前，必须保证ISP效果调试功能已使能.
 */
int tx_isp_tuning_set_hv_flip(ISP_CORE_HVFLIP hvflip);
```

6.12 TAG 分区内容详解

32K	32K 备份	128K RISCV 固件	160K 图像效果文件

TAG 分区如上述图片所示，其中第一段 32K 中按 4K 作为一个小段，分别包含：

```
1、TAG Info            暂时无用
2、Sensor Init Setting   Sensor 初始化 IIC 序列
3、CMDLINE             内核所使用 cmdline 信息
4、Boot info           系统启动信息，与 cmdline 中分区信息保持一致
5、USER info           用户信息，暂时无用
6、AE Table            起始 AE 参数数据
7、FW info             固件信息
```

第二段 32K 为第一个 32K 数据的备份，在第一个 32K 数据损坏的时候，第二个 32K 备份信息起到作用。

第三段大小为 128K，作为 risc-v 固件存放位置。

第四段为图像效果文件，单个效果文件大小最大为 160K。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

6.12.1 TAG 分区数据更新方法

在编译整个固件的时候，可以通过 ./build/build_camera_tag.sh 来生成整个 tag.bin 文件，如果想在设备端更新某个段，可以通过 tag_generator 工具来更新。

先进入 Zeratul_Release/tools/device/tag_generator 目录，执行 make，生成设备上可运行的 tag_generator 工具，并将该工具拷贝到 rootfs 中。

在设备上，通过 ./tag_generator --tag_path=/dev/mtdblock1 --update 再加上对应的需要更新的信息即可。

如更新 riscv_fw.bin 时，只需要执行以下命令即可：

```
./tag_generator --tag_path=/dev/mtdblock1 --update --riscv_fw=riscv_fw.bin
```

如更新起始 AE 参数时，只需要执行以下命令即可：

```
./tag_generator --tag_path=/dev/mtdblock1 --update --aetab=ae_table.bin
```

需要更新图像效果文件时，只需要执行以下命令即可：

```
./tag_generator --tag_path=/dev/mtdblock1 --update --
sensor_setting_mode0=sensor_t23.bin
```

6.12.2 IQ 效果文件更新验证

我们在 SDK 中 tools/make_tag/iq_crc 目录下提供了 CRC 工具源码。可以通过 gcc 编译生成可执行文件，然后对效果文件进行 crc 校验。如下：

```
./crc ../sensor_calibration_bin/jxf35_t23.bin
The crc of ../sensor_calibration_bin/jxf35_t23.bin is:1466787313
```

设备在起来的时候，可以通过 dmesg 信息看到内核打印，其中有以下打印信息，会把当前使用的效果文件的 CRC 值打印出来。然后与 PC 端打印值对比即可。

如果 CRC 值一致，说明使用的是同一个 IQ 文件，如果 CRC 不一致，说明使用的 IQ 文件不是同一个。

```
[    0.051129] @@@@ tx-isp-probe ok(version H20200907a), compiler date=Dec 16 2020 @@@@@
[    0.051594] probe ok ------->jxf35
[    0.051616] jxf35 chip found @ 0x40 (i2c0)
[    0.051620] sensor driver version H20200916a
[    0.051914] Calibration ADDR = a3830000
[    0.056906] Calibration len = 159736
[    0.056958] Calibration len = 159736
[    0.056970] Load Sensor Setting DATE:calibration mode 0 MD5:calibration crc 739411255
[    0.056976] Calibration len = 159736
[    0.059768] NCU: size = 3794725  paddr = 0x5800000
[    0.059789] isp_core_tuning_switch_day_or_night is day:1
[    0.059914] jxf35 stream on
```


Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

7. 常见问题

常见问题请参考《Ingenic_Zeratul_T23_常见问题_FAQ》文档。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.